

2026 €€€ 非专业级别软件能力认证第一轮
(C5P-S1) 提高级 C++语言试题

认证时间：2026 年 9 月 26 日 14:30~16:30

考生注意事项：

- 试题纸共有 13 页，答题纸共有 1 页，满分 100 分。本卷为自制模拟卷，不代表任何官方机构。
- 不得使用任何电子设备（如计算器、手机、电子词典等）或查阅任何书籍资料。

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 下列 Linux 命令中，用于连接并显示文件内容的是（ ）
A. cp
B. cat
C. ping
D. touch
2. 已知某用二叉链表存储的哈夫曼树共有 m 个叶子结点，则该哈夫曼树中总共有多少个空指针域（ ）
A. m
B. $m+1$
C. $2m$
D. $2m+1$
3. 对于一棵包含 n 个结点的完全二叉树，对每个叶子结点向上访问直到根结点，统计路径上经过的结点数之和。总时间复杂度为（ ）
A. $O(n)$
B. $O(n \log n)$
C. $O(n^2)$
D. $O(n^2 \log n)$
4. 小 h 从原点 $(0,0)$ 出发，每一步向右或向上走，到达点 $(5,5)$ ，且始终满足 $x \geq y$ 。满足条件的路径有多少条（ ）
A. 252
B. 90
C. 132
D. 42
5. 用两个栈模拟一个队列，入队时元素统一压入栈 A。正确的出队操作是（ ）
A. B 为空时将 A 全部转移到 B，再从 B 弹出；B 非空时直接从 B 弹出
B. 无论 B 是否为空，都先将 A 全部转移到 B，再从 B 弹出

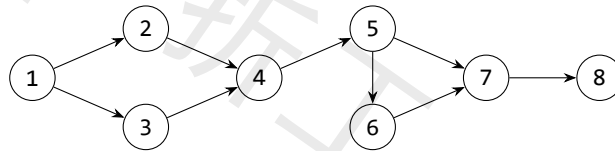
- C. 直接删除 A 的栈底元素
D. 直接从 A 弹出栈顶元素
6. 从集合 $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ 中选出 4 个不同的数, 要求任意两个被选数都不相邻。不同的选法共有 ()
A. 15
B. 20
C. 35
D. 70
7. 在 C++ 中, 关于 `std::set<int>` 与 `std::map<int, int>` 的说法, 正确的是 ()
A. `set` 中的元素可以重复
B. `map` 按插入顺序存储键值对
C. `set` 自动有序, 插入、删除、查找复杂度均为 $O(\log n)$
D. `map` 的键可以重复
8. 关于有向无环图的拓扑排序, 下列说法正确的是 ()
A. 任何有向无环图的拓扑排序都唯一
B. 含有环的有向图也可以进行拓扑排序
C. 若有边 $u \rightarrow v$, 则任一拓扑序中 u 在 v 之前
D. 拓扑排序就是深度优先遍历序列
9. KMP 算法中, `next` 数组的主要作用是 ()
A. 记录模式串失配后应回溯到的位置
B. 记录主串每个字符出现的次数
C. 记录主串的后缀数组
D. 与模式串无关地跳过主串位置
10. 对一个 n 个顶点、 m 条边的带权有向图使用不带堆优化的 Dijkstra 算法, 时间复杂度为 ()
A. $O(n^2)$
B. $O(m \log n)$
C. $O((m+n) \log n)$
D. $O(mn)$
11. 已知 20260913 是质数, 表达式 $(1/2 + 1/3 + \dots + 1/20260912)^{20260913} \bmod 20260913$ 的值为 ()
A. 0
B. 1
C. $2^{20260913} \bmod 20260913$
D. 20260912
12. 设 $p = 20260913$ 为质数。在模 p 意义下,

$$\sum_{i=1}^{p-2} \frac{1}{i(i+1)}$$

等于 ()

- A. 0
- B. 1
- C. 2
- D. $p - 2$

13. 对下图恰好删去两条不同的边, 使得仍存在从 1 到 8 的有向路径。删边方案共有 () 种



- A. 12
 - B. 15
 - C. 18
 - D. 21
14. 1 到 200 之间, 不能被 2、3、7 中任何一个整除的整数共有 () 个
- A. 57
 - B. 58
 - C. 59
 - D. 60
15. 根结点深度为 1, 则一棵深度为 h 的满 k 叉树 ($k > 1$) 的结点数为 ()
- A. $(k^h - 1)/(k - 1)$
 - B. $(k^{(h+1)} - 1)/(k - 1)$
 - C. $k^h/(k - 1)$
 - D. k^h

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填 $\checkmark$ ，错误填 $\times$ ；除特殊说明外，判断题 1.5 分，选择题 3 分，共计 40 分）

(1)

```
01#include <bits/stdc++.h>
02using namespace std;
03const int N = 2e5 + 5;
04int n, pi[N], occ[N], dep[N];
05string s;
06
07int main() {
08    cin >> s;
09    n = s.size();
10    for (int i = 1; i < n; ++i) {
11        int j = pi[i - 1];
12        while (j && s[i] != s[j]) j = pi[j - 1];
13        if (s[i] == s[j]) ++j;
14        pi[i] = j;
15    }
16    for (int i = 0; i < n; ++i) ++occ[pi[i]];
17    for (int i = n - 1; i > 0; --i)
18        occ[pi[i - 1]] += occ[i];
19    for (int i = 0; i <= n; ++i) ++occ[i];
20    long long A = 0, B = 0;
21    for (int i = 1; i <= n; ++i) {
22        dep[i] = dep[pi[i - 1]] + 1;
23        A += occ[i];
24        B += dep[i];
25    }
26    cout << occ[n / 2] << ' ' << dep[n] << ' ' << A
    << '\n';
27}
```

• 判断题

16. 对 $0 \leq i < n$, $pi[i]$ 等于 $s[0..i]$ 的最长真前缀与后缀的公共长度。()
A. 正确
B. 错误
17. 第 18 行循环结束后, $occ[k]$ 等于长度为 k 的前缀在 s 中出现的次数。()

A. 正确

B. 错误

18. 对任意非空输入字符串，程序中的 A 与 B 始终相等。()

A. 正确

B. 错误

• 单选题

19. 输入 aaaaaa 时，程序输出为 ()

A. 3 6 21

B. 4 5 21

C. 4 6 21

D. 4 6 15

20. (4 分) 输入 abacababacabab 时，程序输出为 ()

A. 1 3 30

B. 2 3 32

C. 2 4 32

D. 3 3 34

(2)

```
01#include <bits/stdc++.h>
02using namespace std;
03const int P = 998244353, M = 10;
04int n, m, ban[105], f[2][1 << M];
05vector<int> st;
06
07int main() {
08    cin >> n >> m;
09    for (int i = 1; i <= n; ++i) {
10        string s; cin >> s;
11        for (int j = 0; j < m; ++j)
12            if (s[j] == '#') ban[i] |= 1 << j;
13    }
14    for (int s = 0; s < (1 << m); ++s)
15        if ((s & (s << 1)) == 0) st.push_back(s);
16    f[0][0] = 1;
17    for (int i = 1; i <= n; ++i) {
18        int cur = i & 1, pre = cur ^ 1;
19        memset(f[cur], 0, sizeof(f[cur]));
```

```

20         for (int s : st) {
21             if (s & ban[i]) continue;
22             for (int t : st) {
23                 if (s & t) continue;
24                 f[cur][s] += f[pre][t];
25                 if (f[cur][s] >= P) f[cur][s] -= P;
26             }
27         }
28     }
29     int ans = 0;
30     for (int s : st) {
31         ans += f[n & 1][s];
32         if (ans >= P) ans -= P;
33     }
34     cout << ans << '\n';
35 }

```

假设字符 '.' 表示可选位置，字符 '#' 表示不可选位置。

• 判断题

21. 程序输出从所有可选位置中选取若干位置，使任意两个被选位置不共边的方案数对 P 取模后的结果。()

- A. 正确
- B. 错误

22. 两个只在对角线上相邻的位置可以同时被选取。()

- A. 正确
- B. 错误

23. (2 分) 当 $m=10$ 时, $st.size()$ 等于 144。()

- A. 正确
- B. 错误

• 单选题

24. 记 $V = st.size()$, 则程序的时间复杂度为 ()

- A. $O(nV)$
- B. $O(nV^2 + 2^m)$
- C. $O(n2^{2m})$
- D. $O(n^2V)$

25. 输入为 $n=2, m=3$, 两行均为 ... 时, 程序输出为 ()

- A. 15

B. 16

C. 17

D. 18

26. 输入为 $n=4, m=4$, 四行均为 时, 程序输出为 ()

A. 1134

B. 1198

C. 1234

D. 1296

(3)

```
01#include <bits/stdc++.h>
02using namespace std;
03using ll = long long;
04const ll P = 1000000007;
05const int N = 2e5 + 5;
06int n, q;
07ll a[N];
08struct Node { ll sum, mul, add; } tr[N << 2];
09
10void build(int p, int l, int r) {
11    tr[p].mul = 1;
12    if (l == r) { tr[p].sum = a[l] % P; return; }
13    int m = (l + r) >> 1;
14    build(p << 1, l, m); build(p << 1 | 1, m + 1, r)
15    ;
16    tr[p].sum = (tr[p << 1].sum + tr[p << 1 | 1].sum
17    ) % P;
18}
19void apply(int p, int l, int r, ll k, ll b) {
20    tr[p].sum = (tr[p].sum * k + (r - l + 1) * b) %
21    P;
22    tr[p].mul = tr[p].mul * k % P;
23    tr[p].add = (tr[p].add * k + b) % P;
24}
25void push(int p, int l, int r) {
26    if (l == r) return;
27    int m = (l + r) >> 1;
```

```

25     apply(p << 1, l, m, tr[p].mul, tr[p].add);
26     apply(p << 1 | 1, m + 1, r, tr[p].mul, tr[p].add
    );
27     tr[p].mul = 1; tr[p].add = 0;
28 }
29 void upd(int p, int l, int r, int ql, int qr, ll k,
    ll b) {
30     if (ql <= l && r <= qr) { apply(p, l, r, k, b);
    return; }
31     push(p, l, r);
32     int m = (l + r) >> 1;
33     if (ql <= m) upd(p << 1, l, m, ql, qr, k, b);
34     if (qr > m) upd(p << 1 | 1, m + 1, r, ql, qr, k,
    b);
35     tr[p].sum = (tr[p << 1].sum + tr[p << 1 | 1].sum
    ) % P;
36 }
37 ll qry(int p, int l, int r, int ql, int qr) {
38     if (ql <= l && r <= qr) return tr[p].sum;
39     push(p, l, r);
40     int m = (l + r) >> 1;
41     ll s = 0;
42     if (ql <= m) s = (s + qry(p << 1, l, m, ql, qr))
    % P;
43     if (qr > m) s = (s + qry(p << 1 | 1, m + 1, r,
    ql, qr)) % P;
44     return s;
45 }
46 int main() {
47     cin >> n >> q;
48     for (int i = 1; i <= n; ++i) cin >> a[i];
49     build(1, 1, n);
50     while (q--) {
51         int op, l, r; cin >> op >> l >> r;
52         if (op == 1) {

```



```

53         ll k, b; cin >> k >> b;
54         upd(1, 1, n, 1, r, k, b);
55     } else cout << qry(1, 1, n, 1, r) << '\n';
56 }
57 }

```

操作 1 l r k b 将区间 $[l, r]$ 内的每个数 x 变为 $kx + b$ ；操作 2 l r 输出区间和对 P 取模后的结果。

• 判断题

27. 对同一结点先执行变换 $x \mapsto k_1x + b_1$ ，再执行 $x \mapsto k_2x + b_2$ ，其懒标记表示的合成变换为 $x \mapsto k_1k_2x + b_1k_2 + b_2$ 。（ ）

- A. 正确
- B. 错误

28. 两次操作区间完全相同时，交换这两次操作的先后顺序一定不会改变最终数组。（ ）

- A. 正确
- B. 错误

29. 程序的时间复杂度为 $O((n + q) \log n)$ 。（ ）

- A. 正确
- B. 错误

• 单选题

30. 初始数组为 1 2 3 4，执行 1 2 4 2 1 后再询问 2 1 4，输出为（ ）

- A. 20
- B. 21
- C. 22
- D. 23

31. 初始数组为 1 2 3 4 5，依次执行 1 1 4 2 1、1 3 5 1 3、2 2 5，最后一次操作输出为（ ）

- A. 32
- B. 33
- C. 35
- D. 37

32. (4 分) 初始数组为 2 1 3 5 4 6，依次执行 1 2 6 2 1、1 1 4 3 2、1 3 5 1 4、2 2 6，最后一次操作输出为（ ）

- A. 97
- B. 101
- C. 103
- D. 107

三、完善程序（单选题，每小题 3 分，共计 30 分）

(1) (有序序列的第 k 小和) 有两个长度均为 n 的单调不降序列 a 和 b 。从两个序列中各取一个数相加，共得到 n^2 个和，求这些和中第 k 小的值。输入满足 $1 \leq n \leq 10^5$, $1 \leq k \leq n^2$ ，元素为非负整数且答案不超过 10^9 。

```
01#include <bits/stdc++.h>
02using namespace std;
03const int N = 100005;
04int n, a[N], b[N];
05long long k;
06int upper_bound_index(int *a, int n, long long x) {
07    int l = 0, r = ①;
08    while (l < r) {
09        int mid = (l + r) >> 1;
10        if (②)
11            r = mid;
12        else
13            l = mid + 1;
14    }
15    return ③;
16}
17long long count_le(long long sum) {
18    long long cnt = 0;
19    for (int i = 0; i < n; ++i)
20        cnt += upper_bound_index(b, n, sum - a[i]);
21    return cnt;
22}
23long long solve() {
24    long long l = ④, r = ⑤;
25    while (l < r) {
26        long long mid = (l + r) >> 1;
27        if (count_le(mid) >= k)
28            r = mid;
29        else
30            l = mid + 1;
31    }
```

```

32     return l;
33 }
34 int main() {
35     ios::sync_with_stdio(0);
36     cin.tie(0);
37     cin >> n >> k;
38     for (int i = 0; i < n; ++i) cin >> a[i];
39     for (int i = 0; i < n; ++i) cin >> b[i];
40     cout << solve();
41 }

```

33. ①处应填 ()
 A. n B. n-1
 C. x D. 0
34. ②处应填 ()
 A. $a[mid] > x$ B. $a[mid] \geq x$
 C. $a[mid] < x$ D. $a[mid] \leq x$
35. ③处应填 ()
 A. 1 B. l+1
 C. r-1 D. n-1
36. ④处应填 ()
 A. $a[0]+b[0]$ B. $a[1]+b[1]$
 C. 0 D. $a[n]+b[n]$
37. ⑤处应填 ()
 A. $a[n-1]+b[n-1]$ B. $a[n]+b[n]$
 C. $2*N$ D. n-1

(2) (最小生成树) 给定一个有 n 个结点、 m 条带权无向边的图, 求其最小生成树的边权和; 若图不连通则输出 Impossible。输入满足 $1 \leq n \leq 2 * 10^5$, $1 \leq m \leq 4 * 10^5$ 。

```

01 #include <bits/stdc++.h>
02 using namespace std;
03 const int N = 2e5 + 5, M = 4e5 + 5;
04 struct Edge {
05     int u, v, w;
06     bool operator < (const Edge &other) const {
07         return w < other.w;
08     }

```

```

09} e[M];
10int n, m, fa[N];
11int find(int x) {
12    return fa[x] == x ? x : fa[x] = find(fa[x]);
13}
14int main() {
15    ios::sync_with_stdio(0);
16    cin.tie(0);
17    cin >> n >> m;
18    for (int i = 1; i <= n; ++i)
19        fa[i] = ①;
20    for (int i = 0; i < m; ++i)
21        cin >> e[i].u >> e[i].v >> e[i].w;
22    sort(e, e + ②);
23    long long ans = 0;
24    int used = 0;
25    for (int i = 0; i < m; ++i) {
26        int x = find(e[i].u), y = find(e[i].v);
27        if (③)
28            continue;
29        fa[x] = ④;
30        ans += e[i].w;
31        ++used;
32    }
33    if (used != ⑤)
34        cout << "Impossible";
35    else
36        cout << ans;
37}

```

38. ①处应填 ()

- A. i B. 0
C. n D. fa[i-1]

39. ②处应填 ()

- A. m B. m-1
C. M D. n

40. ③处应填 ()

- A. `x == y` B. `x != y`
C. `x < y` D. `e[i].u == e[i].v`

41. ④处应填 ()

- A. `x` B. `y`
C. `e[i].v` D. `fa[y]`

42. ⑤处应填 ()

- A. `n` B. `n-1`
C. `m` D. `used+1`